Interior-Gen

System Architecture & Presentation Walkthrough

AI Interior Redesign + AI-Assisted House Planning

Backend: FastAPI + SQLAlchemy | Frontend: Vanilla JS + Tailwind + GSAP

Self-hosted models on Kaggle + OpenAI gpt-image-1 | AWS S3 | Clerk Auth

1. What Interior-Gen Is

Interior-Gen is a B2B AI tool with two independent generation products under one account system and one billing system:

- Room Redesign - upload a photo of a real room, get back 3 visually distinct redesigns (Economical / Mid / Premium) that preserve the real walls, windows, doors, layout and camera angle - plus an itemized, real-priced materials list per tier.
- Build a House - enter plot dimensions + a room program (floors/bedrooms/bathrooms/extras), get back a deterministically-computed, dimensionally-accurate floor plan (PNG + real AutoCAD .dxf file), a feasibility check, an AI 'Concept Layout' visual, and an AI-generated exterior elevation render.

Both products share the same account system (Clerk), the same subscription/quota system (Free / Pro / Studio), and the same storage backend (AWS S3) - but each has its own database table, its own pipeline module, and its own AI models, because the fields and stages genuinely differ.

2. Tech Stack At A Glance

Backend

Language / framework	Python 3.13, FastAPI
Database	SQLite (local dev) / Postgres via Neon (production)
ORM	SQLAlchemy
Async work	FastAPI BackgroundTasks + client polling (no message queue yet)
Object storage	AWS S3 (via boto3) - image bytes, never stored in the DB
Auth	Clerk (hosted signup/login/Google sign-in, Bearer-token sessions)

Frontend

Stack	Plain HTML + vanilla JavaScript - NO build step, NO framework
Styling	Tailwind CDN (inline config per page) + a custom 'Sage & Linen' theme
Motion	GSAP + ScrollTrigger (loaded via CDN)
Fonts	Ranade (headings, self-hosted) + Poppins (body, Google Fonts)

AI / Generation

Paid image model	OpenAI gpt-image-1 (instruction-following image EDIT model)
Free text model	Google Gemini 2.5 Flash (room/plot descriptions, structured room layouts, materials pricing synthesis)
Self-hosted models	3 separate models, each on its own Kaggle notebook + Cloudflare tunnel (details in Section 5)
Pricing data	SerpApi (real product search) + Gemini (price synthesis from real results)

Payments (current state)

Status	Manual, honor-system stopgap - NOT a real payment integration yet
Flow today	User sends money via JazzCash to a real account shown in-app; upgrade is done manually, there is no automatic account flip yet
Researched for later	Safepay (recommended), PayFast, JazzCash API - none committed yet

3. The Three Architectural Seams

The whole backend is deliberately built around three swappable 'seams' - interfaces that let any model, storage backend, or prompt be swapped out without touching the rest of the system. This is the single most important architectural idea to mention when presenting.

1. Provider seam (app/providers/)

Every single AI model call - image generation, room description, materials pricing, floor plans, elevation renders - goes through one abstract Provider interface. A HybridProvider class is the real composition root: it decides, per request, which concrete provider (OpenAI, Kaggle, Gemini) actually handles the call, and transparently falls back to OpenAI if a self-hosted model fails. Swapping or adding a model is a change inside this one folder only.

2. Storage seam (app/storage/)

Image bytes always go through a Storage interface (local disk for dev, AWS S3 in production). The database NEVER stores image bytes - only metadata and S3 keys. Keys are organized as users/{user}/roomRedesign/... and users/{user}/buildAHouse/... so a bucket can be browsed per person, per feature.

3. Prompt seam (app/pipeline/prompts.py, house_prompts.py)

Every prompt sent to an image model is assembled from structured data (a dictionary of material/lighting/style specs per tier), not hand-typed strings scattered through the code. This is what lets 'Economical vs Mid vs Premium' be a real, structured cost methodology (paint -> flooring -> lighting -> feature walls -> decor) instead of vague adjectives.

4. Room Redesign - End-to-End Flow

What happens between a user clicking 'Generate' and seeing 3 finished rooms:

- 1. Upload - user uploads a room photo + optional style notes, city (for pricing), and real room dimensions. Stored in S3, a Project row created in the database.
- 2. Room description - Gemini looks at the photo once and writes a structured description (room type, walls/windows/doors, proportions/depth) that gets reused by every later prompt.
- 3. Tier notes - Gemini also looks at the SAME photo and writes short, room-specific instructions per tier (e.g. 'repaint over visible water stains' for the Economical tier), so a badly damaged room actually looks renovated at the cheap tier too.
- 4. Materials pricing kicked off in parallel - as soon as the description is ready, a background thread pool starts searching real prices (SerpApi) for each tier's materials list (flooring, paint, lighting, ceiling, feature wall, decor) WHILE the images are still generating, so pricing never adds extra wait time.
- 5. Image generation - all 3 tiers (Economical / Mid / Premium) are generated concurrently. Each tier's prompt is a full natural-language paragraph built from the structured tier spec + the room description + the tier note + the user's own style text, with an explicit 'do not change the room's structure/camera angle/depth' instruction baked into every tier.
- 6. Model choice - the actual image model used is either the self-hosted Kaggle model or OpenAI gpt-image-1, decided per-request by the provider seam (and shown to the user afterward: 'Generated using our model' or 'Generated using OpenAI').
- 7. Results - all 3 images + the original + the room description + per-tier materials lists land in S3 and the database; the frontend polls the project's status until everything is ready and renders the results grid with a lightbox and a materials popup per tier.

5. Build a House - End-to-End Flow

This is the more complex of the two products - it runs a genuinely deterministic architectural engine, not just an AI image call.

- 1. Input - plot length/width/unit, an optional entrance facing (N/S/E/W, defaults South), floors/bedrooms/bathrooms via dropdowns, free-text 'extras' (e.g. 'garage, dirty kitchen'), and an OPTIONAL plot photo (the photo is decorative context only - the floor plan never needs it).
- 2. Plot analysis - Gemini looks at the photo (if given) and writes a short description; best-effort, skipped entirely if no photo was uploaded.
- 3. Room program - Gemini turns the floor/bedroom/bathroom counts + extras text into a structured list of rooms per floor, following real architectural convention (garage/kitchen only on the ground floor, bathrooms paired sensibly, etc). A deterministic garage-count parser and an unrequested-garage stripper both run on top of this, since an LLM's room choices are treated as a suggestion, never as ground truth.
- 4. Deterministic floor layout - the real core of this feature (app/pipeline/floor_layout.py, zero AI involved). Rooms are sliced into real, non-overlapping rectangles that: guarantee every room a real minimum usable size, cap oversized rooms, zone public rooms (garage/entry/living/kitchen/dining) at the front and private rooms (bedrooms/bathrooms) at the back relative to the chosen entrance facing, pair each bedroom with its own ensuite bathroom, build a real hallway corridor through the private zone, and give the garage a real car-fitting width instead of just the right area.
- 5. Feasibility check - a pure-math gate (no AI) that sums every room's real minimum area + a circulation/wall overhead and compares it against the plot's actual buildable footprint. An infeasible request blocks the floor-plan/AI-visual stages with a plain explanation instead of silently drawing something that doesn't fit.
- 6. Blueprint rendering - the exact same computed rectangles are drawn twice: once as a labeled, furnished PNG floor plan (solid poche walls, doors, windows, staircases, furniture symbols - drawn with Pillow, zero AI), and once as a REAL AutoCAD .dxf file (via the ezdxf library) with real wall thickness and real door/window openings - both are always in perfect agreement because they come from the identical geometry.
- 7. Concept Layout (AI, supplementary only) - the SAME computed geometry is rendered as a clean black-and-white edge map and sent to a friend-hosted Kaggle SDXL + ControlNet model as a conditioning image, so that model traces our real floor plan instead of hallucinating its own. This runs on a fire-and-forget background thread and does not block the rest of the project from finishing - it is explicitly labeled 'Concept Layout - not a precise blueprint' in the UI, never presented as the accurate deliverable.
- 8. Exterior elevation render - a photorealistic front-of-house image, generated either by a self-hosted Kaggle text-to-image model (see Section 6c) or OpenAI gpt-image-1 editing the plot photo, depending on which backend is configured/chosen.
- 9. Results - the elevation render, the .dxf download, the labeled floor-plan PNGs, and the AI Concept Layout image are all shown together, each clearly labeled for what it actually is.

6. The Three Self-Hosted Kaggle Notebooks

All three self-hosted models run the same basic way: a free Kaggle GPU (T4) notebook loads a model once, wraps it in a small FastAPI server, and exposes it to the internet through a free Cloudflare quick tunnel. The main app talks to whichever tunnel URL is currently live, over plain HTTPS, using a submit-a-job-then-poll-for-the-result pattern (because the free tunnel kills any single request that stays open past about 100 seconds).

6a. Room Redesign model

Used for	The 3-tier Room Redesign image generation (self-hosted alternative to OpenAI)
Model type	Stable Diffusion img2img + ControlNet (depth-conditioned)
Input	The user's real uploaded room photo, converted to a depth map for ControlNet
Why ControlNet	Locks the model onto the real room's structure/geometry so walls, windows and camera angle are preserved, not reinvented
Code seam	app/providers/kaggle.py

6b. AutoCAD / Concept Layout model

Used for	The supplementary 'Concept Layout' visual on Build a House
Model type	Stable Diffusion XL + ControlNet (canny-edge conditioned)
Input	A black-and-white edge-map image of OUR OWN computed floor-plan geometry (not the plot photo) - so the AI traces a real, correct layout instead of guessing one
Important honesty point	This is still a pixel/image model, not a real CAD/vector engine - it cannot be relied on for exact dimensions, which is exactly why the deterministic PNG + real .dxf export exist as the authoritative output
Code seam	app/providers/kaggle_autocad.py

6c. Elevation (exterior render) model

Used for	Build a House's photorealistic front-elevation render
Model type	RealVisXL_V4.0 (Stable Diffusion XL variant) - pure TEXT-TO-IMAGE, no ControlNet, no input photo needed at all
Input	A short, minimal features prompt + structured signals: floor count, and a real True/False garage flag (so the model neither invents nor omits a garage by accident)
Reference file	kaggle_notebooks/elevation_server.py - the one notebook in this project that is actually version-controlled in the repo, since it was hand-written for this project
Code seam	app/providers/kaggle.py (generate_house_render)

> *Presentation talking point: all three notebooks follow the exact same infrastructure pattern (Kaggle GPU + FastAPI + Cloudflare tunnel + submit/poll), which is worth saying out loud - it shows this wasn't three one-off hacks, it's one repeatable, understood pattern applied three times to three different models.*

7. File Map - What Lives Where

Backend core

app/main.py	All FastAPI routes - upload endpoints, polling endpoints, auth-gated project CRUD, pricing/plan endpoints
app/models.py	Database tables - Project, HouseProject, UserPlan
app/config.py	All environment/settings, including which provider each feature uses
app/db.py	Database engine + startup migrations
app/plans.py	Free/Pro/Studio quota logic

Room Redesign pipeline

app/pipeline/generate.py	Orchestrates the whole Room Redesign run
app/pipeline/prompts.py	The structured tier specs + prompt builder

Build a House pipeline

app/pipeline/generate_house.py	Orchestrates the whole Build a House run
app/pipeline/floor_layout.py	The deterministic room-slicing engine - the real core logic
app/pipeline/room_specs.py	Real-world min/max room sizes per room type
app/pipeline/feasibility.py	The pure-math feasibility gate
app/pipeline/blueprint_svg.py	Draws the labeled, furnished floor-plan PNG
app/pipeline/blueprint_dxf.py	Exports the same geometry as a real AutoCAD .dxf file
app/pipeline/conditioning_image.py	Renders the edge-map sent to the AI Concept Layout model
app/pipeline/house_prompts.py	Prompt builders for the elevation render + edit-style renders

Providers (the AI model seam)

app/providers/hybrid.py	The composition root - decides which real provider handles each call
app/providers/openai.py	OpenAI gpt-image-1 image generation/editing
app/providers/gemini.py	Gemini text calls - descriptions, room layouts, materials pricing
app/providers/kaggle.py	Room Redesign model + Elevation model (self-hosted)
app/providers/kaggle_autocad.py	AI Concept Layout model (self-hosted)

Frontend

static/index.html	The whole app - Home / Room Redesign / Build a House / Pricing tabs
static/app.js	All frontend logic - uploads, polling, rendering results, the pricing UI
static/components.css	Styles for JS-injected HTML fragments (result cards, modals, etc.)

8. Accounts, Database & Storage

Authentication - Clerk

Sign-up, login, Google sign-in and session handling are entirely owned by Clerk (a third-party auth provider) - the app never sees a password. The frontend gets a Bearer token from Clerk and sends it on every request; the backend verifies that token and only ever knows the user's Clerk ID (no local username/password table exists at all).

Database

SQLite locally, Postgres (hosted on Neon) in production. Three main tables: Project (Room Redesign runs), HouseProject (Build a House runs), UserPlan (subscription/quota state). Only metadata and S3 keys are stored here - never image bytes.

Storage - AWS S3

Every image (uploads and generated results) lives in a single S3 bucket, organized per user, per feature: users/{clerk-id}_{name}/roomRedesign/... and users/{clerk-id}_{name}/buildAHouse/... This makes it possible to browse exactly what one person has generated directly in the bucket.

9. Subscription & Pricing System

A real, working quota system exists today - here is exactly what's live vs. what's still manual:

Live today

- Free / Pro / Studio plans with real monthly quotas, enforced on Room Redesign generations.
- A 1-free-generation trial for anonymous (not-logged-in) visitors, for both products.
- Per-request choice between the self-hosted model and OpenAI - available to anonymous trial users and Pro/Studio, silently locked to the self-hosted model for Free plan users.
- A full Pricing tab with plan comparison, and a manual JazzCash payment modal showing account details for Pro/Studio upgrades.

Still manual / not yet built

- No real payment processor is wired in - JazzCash today is pay-then-message-us, not automatic.
- No way yet to actually flip a user's plan automatically after payment - this is done by hand.
- Build a House has NO quota enforcement yet - it is fully unmetered on every plan today, because its render step currently depends on OpenAI (paid) rather than a free self-hosted model.

10. Presentation Talking Points

Short, quotable lines to use when presenting each part of the system:

- "Every AI model call in this app goes through one swappable interface - we can add or replace a model without touching the rest of the product."
- "The floor plan you see isn't drawn by an AI guessing - it's computed with real geometry, real minimum room sizes, and a real feasibility check. The AI 'Concept Layout' card is a bonus visual on top of that, not the source of truth."
- "We can export a genuine AutoCAD .dxf file that opens in real CAD software, because the same exact geometry that draws the picture also drives the file export - they can never disagree."
- "We run three different self-hosted AI models, each on its own free Kaggle GPU, using the same repeatable infrastructure pattern - so hosting is effectively free while we validate the product."
- "Every generation tells the user honestly which model actually produced it - our self-hosted model or OpenAI - we never hide or fake that."
- "The subscription system is real and enforced today for Room Redesign - Free, Pro and Studio tiers with real monthly quotas - payment collection is currently a manual, honest stopgap while we finalize a Pakistan-compatible payment processor."